

ENP6 · UNAM

ETE Computación · Curso Web

• GIT Y GITHUB

Git y GitHub

El cuaderno del programador

- EN ESTE TEMA APRENDERÁS

Al terminar este tema podrás...

- **RECORDAR** — **definir** control de versiones, repositorio, commit, rama, merge y los estados de un archivo en Git
- **COMPRENDER** — **explicar** la diferencia entre Git, GitHub y GitLab, y por qué un controlador de versiones es indispensable en cualquier proyecto
- **APLICAR** — **ejecutar** el flujo completo `init` → `add` → `commit` → `push` y publicar un repositorio en GitHub
- **ANALIZAR** — **interpretar** el historial de un repositorio y el ciclo de vida de cada archivo usando `git log` y `git status`
- **APLICAR** — **crear** ramas, contribuir en equipo y resolver conflictos de merge

El recorrido

INTRO	• ¿POR QUÉ GIT?
B0-B1	Imagina esto • Historia • BitKeeper
HITO 1	Git ≠ GitHub • GitLab
PARTE 1	• INSTALAR Y CONFIGURAR
B2	git --version • git config • GitHub signup
ACT	Configura tu entorno
PARTE 2	• TU PRIMER REPOSITORIO
HITO 2	git init • .git/ • .gitignore
ACT	Tu primer repositorio
PARTE 3	• CICLO DE VIDA
HITO 3	add • commit • Conventional Commits
ACT	Tu primer commit

■ parte ■ HITO • ejecutar en terminal ■ actividad • entregable

PARTE 4	• CONECTAR CON GITHUB
HITO 4	remote add • push • pull • tokens
ACT	Subir a GitHub
PARTE 5	• TRABAJO EN EQUIPO
B6	Clonar • commit en equipo • git log
PARTE 6	• RAMAS Y MERGE
HITO 5	Tu rama personal • checkout -b
HITO 6	Tu primer merge a main
ACT	Tu rama → merge a main
CIERRE	
—	Cheat sheet completa • recursos • entrega

- IMAGINA ESTO

“Llevas 12 días trabajando en tu proyecto. Es el día antes de la entrega final. Abres tu computadora y la carpeta está vacía.”

PREGUNTA AL GRUPO

¿Qué harías?

¿Te suena familiar?

 proyecto/
 proyecto_final/
 proyecto_final_ESTESÍ/
 proyecto_ENTREGAR_MÑANA/

 proyecto_v2/
 proyecto_final_BUENO/
 proyecto_final_ahora_sí_de_verdad/
 proyecto_ENTREGAR_MÑANA_este/

Todos lo hemos hecho.

- TRABAJO EN EQUIPO

5 personas • 1 archivo • WhatsApp

Sofía

proyecto_v3.zip

Diego

proyecto_FINAL.zip

Karla

proyecto_mio.zip

Miguel

proy_bueno.zip

Ana

proyecto.zip

¿Cuál versión es la buena?

¿Quién borró la función que funcionaba?

¿Cómo regresamos a como estaba ayer?

Intro

¿Por qué Git?

- CONCEPTO

Control de versiones

Sistema que **registra los cambios** de archivos a lo largo del tiempo y permite **recuperar versiones anteriores**.

LO CONOCES COMO...

Historial de versiones de Google Docs.

GIT ES ESO PERO...

para **cualquier archivo**, **offline**, con **superpoderes** de equipo.

- HISTORIA · ¿POR QUÉ EXISTE GIT?

El drama de BitKeeper



Moraleja: a veces la mejor herramienta del mundo **nace de un berrinche bien dirigido.**

- HISTORIA

2005 • Linus Torvalds • 2 semanas

- El kernel de Linux necesitaba un VCS distribuido y gratuito
- Linus lo construyó en aproximadamente 2 semanas
- Hoy lo usa más del 90% de los proyectos de software

PROYECTOS QUE LO USAN

LINUX

ANDROID

WINDOWS

VS CODE

REACT

ESTE CURSO

- MOTIVACIÓN

No es opcional en el mundo real

+100M

Desarrolladores en GitHub

+420M

Repositorios públicos

90%+

Proyectos de software que usan Git

En este curso, cada día hacen commits. Al terminar tienen un **portafolio real en GitHub** que dice más que cualquier calificación.

Intro ¿Por qué Git?

La confusión más común

Git

- Herramienta local
- Vive en tu computadora
- Línea de comandos
- Open source • gratis

≠

GitHub

- Plataforma en internet
- Vive en la nube
- Interfaz web
- Gratis con planes de pago

Puedes usar **Git** sin **GitHub**. **No** puedes usar **GitHub** sin **Git**.

- ANALOGÍA

Una forma de pensarlo

Git

Tu cuaderno de bitácora.

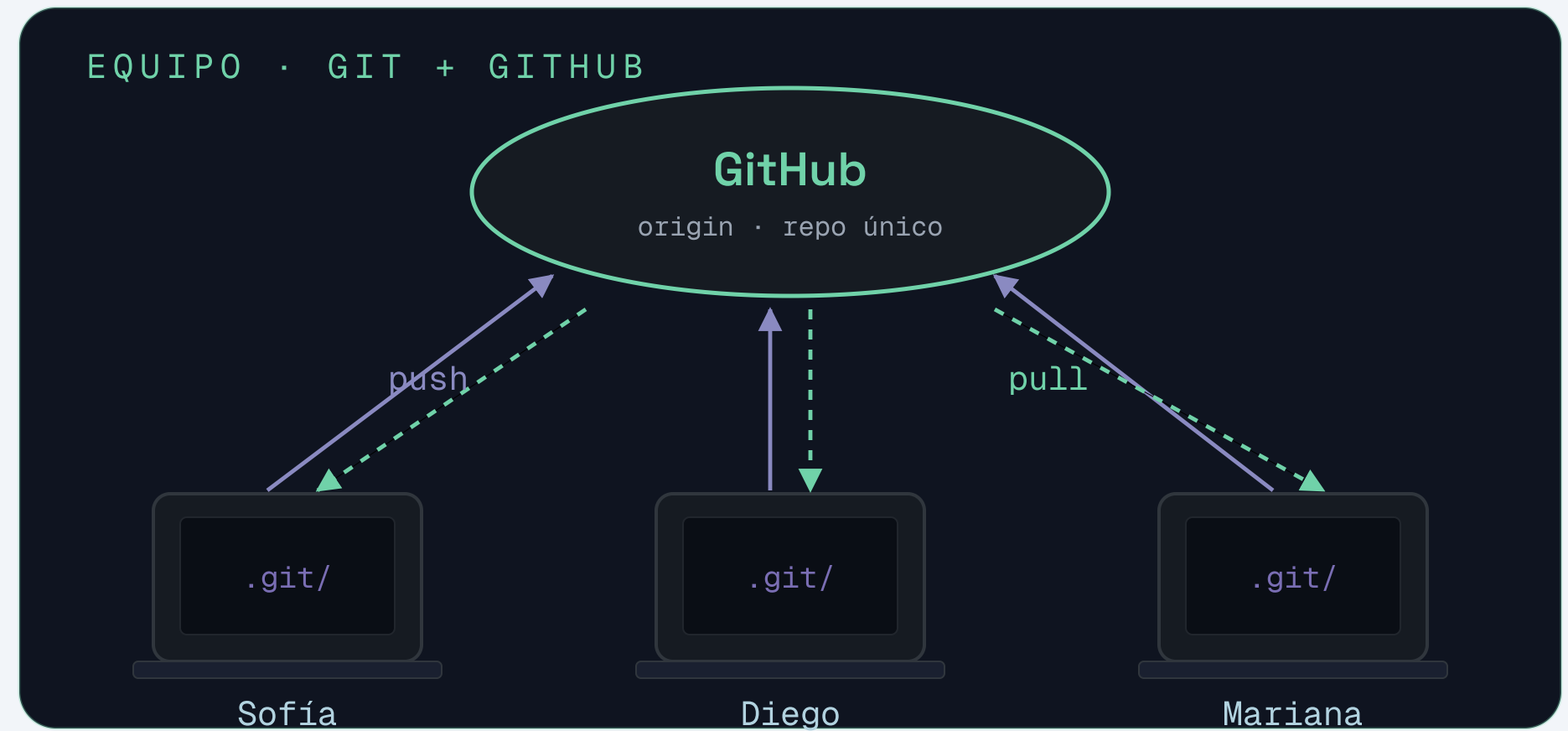
En tu mochila, siempre contigo. Funciona offline. Solo tú lo lees.

GitHub

La biblioteca donde lo compartes.

El mundo puede leerlo, copiarlo, contribuir.
Necesita internet.

Solo te basta Git. En equipo, necesitas GitHub.



Git vive en cada laptop • GitHub es el punto único donde todos sincronizan.

- PLATAFORMA

GitHub

- **Hospedaje** de repositorios remotos
- **Colaboración:** Issues y Pull Requests
- **Automatización:** GitHub Actions
- **Publicación:** GitHub Pages — sitios web gratis
- Adquirido por **Microsoft en 2018**

- ALTERNATIVA

GitLab — la alternativa

- [Open source](#) — instalable en tu propio servidor
- Muy usado en [empresas, gobiernos e instituciones educativas](#)
- [CI/CD](#) integrado desde el inicio
- [UNAM](#) y muchas instituciones lo usan

- COMPARATIVA

¿Cuál es cuál?

	GIT	GITHUB	GITLAB
Tipo	Herramienta local	Plataforma nube	Nube o local
Creado por	Linus Torvalds	GitHub Inc.	GitLab Inc.
Open source	Sí	No	Sí
Gratuito	Sí	Parcial	Parcial
Requiere internet	No	Sí	Depende
Usamos en el curso	✓	✓	—

PARTE 1

Instalar y configurar Git

- PARTE 1 • OBJETIVOS DE APRENDIZAJE

En esta parte aprenderás a...

- RECORDAR — **listar** los comandos necesarios para instalar y verificar Git
- COMPRENDER — **explicar** qué registra Git sobre tu identidad y por qué cada commit la incluye
- APLICAR — **instalar** Git, configurar nombre y correo globales, y crear tu cuenta en GitHub

En esta sección...

- Verificar si [Git](#) ya está instalado en tu computadora
- Instalarlo si es necesario (Homebrew / apt / winget)
- Configurar tu [identidad global](#): nombre y correo
- Crear tu cuenta en [GitHub](#)

Cada commit lleva tu firma

Cada vez que haces un commit, Git registra **quién lo hizo**. Esa información aparece en el historial del proyecto **para siempre**.

```
$ git log --oneline
a3f8c21 Sofía López   Agrega formulario de inscripción
f91d340 Sofía López   Primer commit: agrega README
```

El nombre importa.

¿Tienes Git instalado?

Abre la terminal — [Git Bash](#) en Windows, [Terminal](#) en Mac/Linux.

```
$ git --version
```

RESULTADO ESPERADO

```
git version 2.43.0
```

Si no aparece, tendrás que instalarlo según tu sistema operativo — en la siguiente diapositiva están los comandos.

Instalar Git desde la terminal

Windows

Desde **PowerShell** o **cmd**, con winget incluido en Windows 10/11:

```
PS> winget install --id Git.Git -e --source winget
```

Alternativa: choco install git (Chocolatey) o instalador desde git-scm.com/download/win

macOS

Lo más rápido — Command Line Tools:

```
$ xcode-select --install
```

Con **Homebrew** (versión más nueva):

```
$ brew install git
```

Si no tienes Homebrew: brew.sh

Linux

Debian / Ubuntu / Mint:

```
$ sudo apt update  
$ sudo apt install git
```

Fedora / RHEL:

```
$ sudo dnf install git
```

Arch / Manjaro:

```
$ sudo pacman -S git
```

Después de instalar, **cerrar y volver a abrir la terminal** y verificar con `git --version`.

Dile a Git quién eres

Escribe con **tus datos reales**:

```
$ git config --global user.name "Tu Nombre Completo"
$ git config --global user.email "[email protected]"
```

Usar el **mismo correo** que vas a usar en GitHub.

Confirma que quedó bien

• • • ~

```
$ git config --list
```

BUSCA ESTAS LÍNEAS EN LA SALIDA

```
user.name=Tu Nombre Completo  
user.email=[email_protected]
```

Si algo está mal: repetir **git config --global** con el dato correcto. Sobrescribe lo anterior.

Tu cuenta en GitHub

- 1 Abre github.com en el navegador
- 2 Clic en [Sign up](#)
- 3 Correo → contraseña → nombre de usuario → verificar correo → plan [Free](#)

Tu **nombre de usuario es público y permanente** — elígelo bien.

Recomendación: nombre real o profesional, sin caracteres especiales ni fechas de nacimiento.

Configuración inicial — referencia rápida

```
# Verificar versión de Git instalada  
git --version  
  
# Configurar identidad (una sola vez por computadora)  
git config --global user.name "Tu Nombre"  
git config --global user.email "[email_protected]"  
  
# Verificar configuración  
git config --list
```

Configura tu entorno

QUÉ SE PIDE

- Verifica tu versión: `git --version`
- Configura nombre y correo con `git config --global`
- Verifica con `git config --list`
- Crea tu cuenta en [GitHub](#) si aún no tienes

QUÉ SE ENTREGA

- Captura de terminal mostrando `git --version` y `git config --list` con tu nombre y correo
- URL de tu perfil de [GitHub](#)

¿Qué cubrimos?

Conceptos clave

- Git registra **quién** hizo cada cambio — por eso configuramos nombre y correo antes de cualquier commit
- La identidad `--global` aplica a todos los repositorios de esa computadora
- **GitHub** es la plataforma remota; **Git** es la herramienta que vive en tu computadora

Comandos vistos

```
git --version
git config --global user.name "Tu Nombre"
git config --global user.email "tu@correo"
git config --list
```

Checkpoint — ¿puedes explicarle a alguien de tu equipo para qué sirve cada comando?

PARTE 2

Tu primer repositorio

- PARTE 2 · OBJETIVOS DE APRENDIZAJE

En esta parte aprenderás a...

- RECORDAR — **nombrar** las cuatro capas de Git: Working Directory, Staging Area, Repositorio local y Repositorio remoto
- COMPRENDER — **explicar** qué crea `git init` y qué archivos debe excluir un `.gitignore`
- APLICAR — **crear** un repositorio local, explorar la carpeta `.git/` y configurar el archivo `.gitignore`

En esta sección...

- Entender qué es un `repositorio` y sus 4 zonas
- Crear tu primer repositorio con `git init`
- Explorar qué hay dentro de la carpeta `.git/`
- Configurar `.gitignore` para excluir archivos del seguimiento

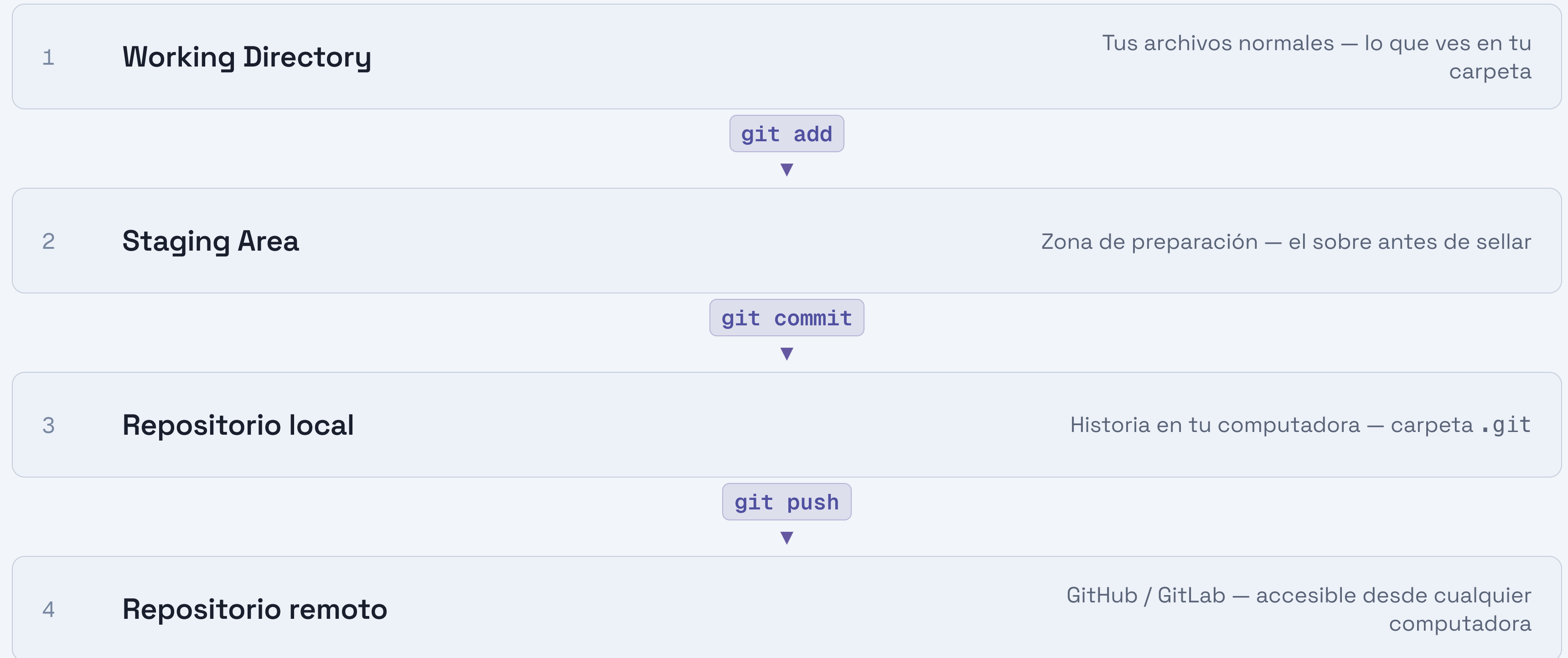
Repositorio

Una **carpeta con superpoderes**. Git registra cada cambio que ocurre dentro de ella.

CONTIENE

- El código del proyecto
- El historial completo: **quién cambió qué y cuándo**
- La configuración del proyecto — carpeta oculta **.git/**

Así funciona Git por dentro • las 4 capas



Working Directory

1 Working Directory

`git add`

2 Staging Area

`git commit`

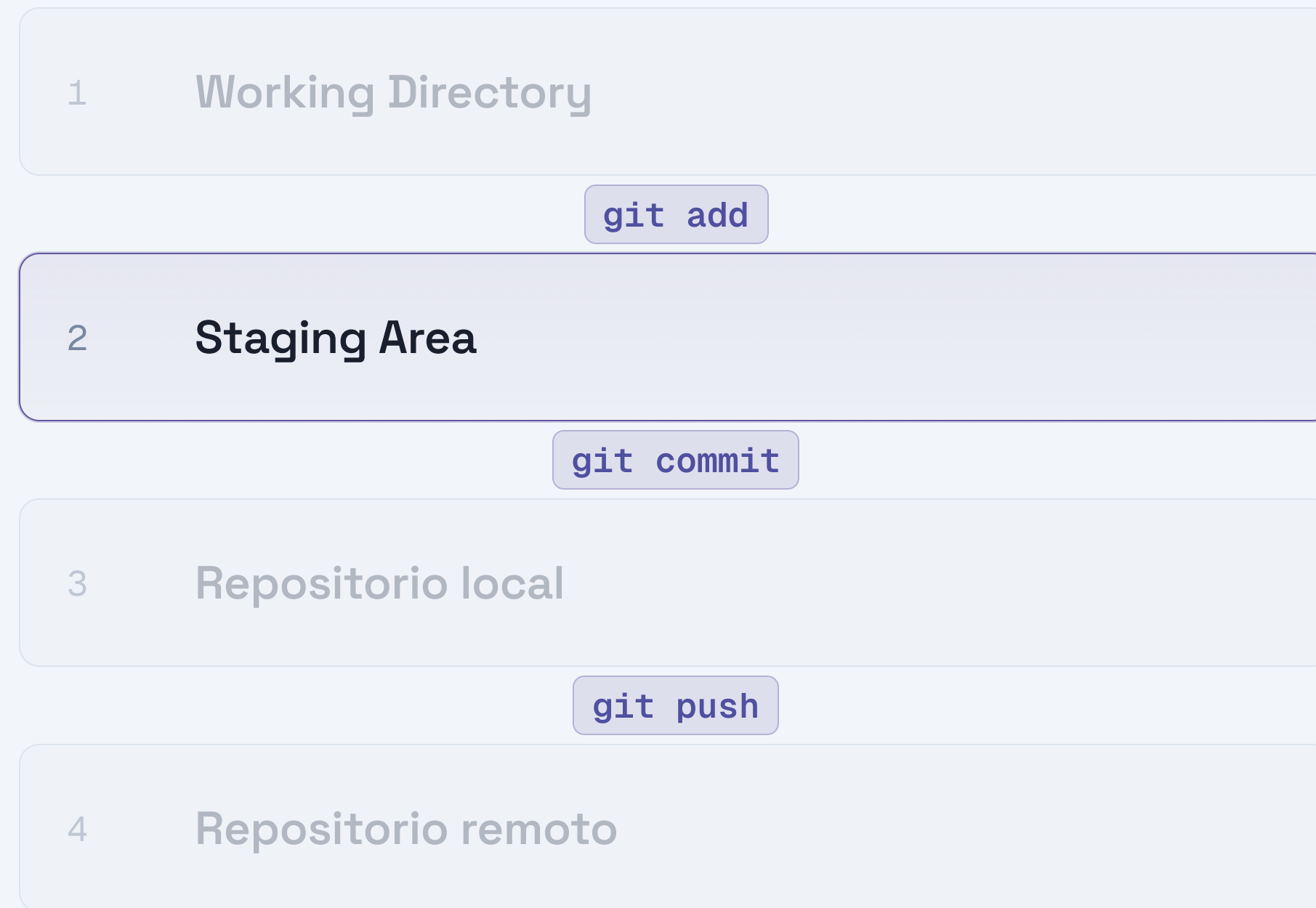
3 Repositorio local

`git push`

4 Repositorio remoto

- Tu carpeta de trabajo normal
- Los archivos que ves, creas y editas
- Git los observa pero aún no los registra
- Analogía: el borrador sobre tu escritorio

Staging Area



- Zona de preparación — archivos listos para el próximo commit
- `git add` los mueve aquí
- Puedes agregar algunos archivos y dejar otros fuera
- Analogía: los documentos que ya metiste al sobre, pero aún no lo sellaste

Repositorio local

1 Working Directory

`git add`

2 Staging Area

`git commit`

3 Repositorio local

`git push`

4 Repositorio remoto

- La base de datos de Git en tu computadora
- Carpeta oculta `.git/` dentro de tu proyecto
- Guarda toda la historia — quién, qué, cuándo
- Funciona **completamente sin internet**
- No borres `.git/` — eso es el repositorio

Repositorio remoto

1 Working Directory

`git add`

2 Staging Area

`git commit`

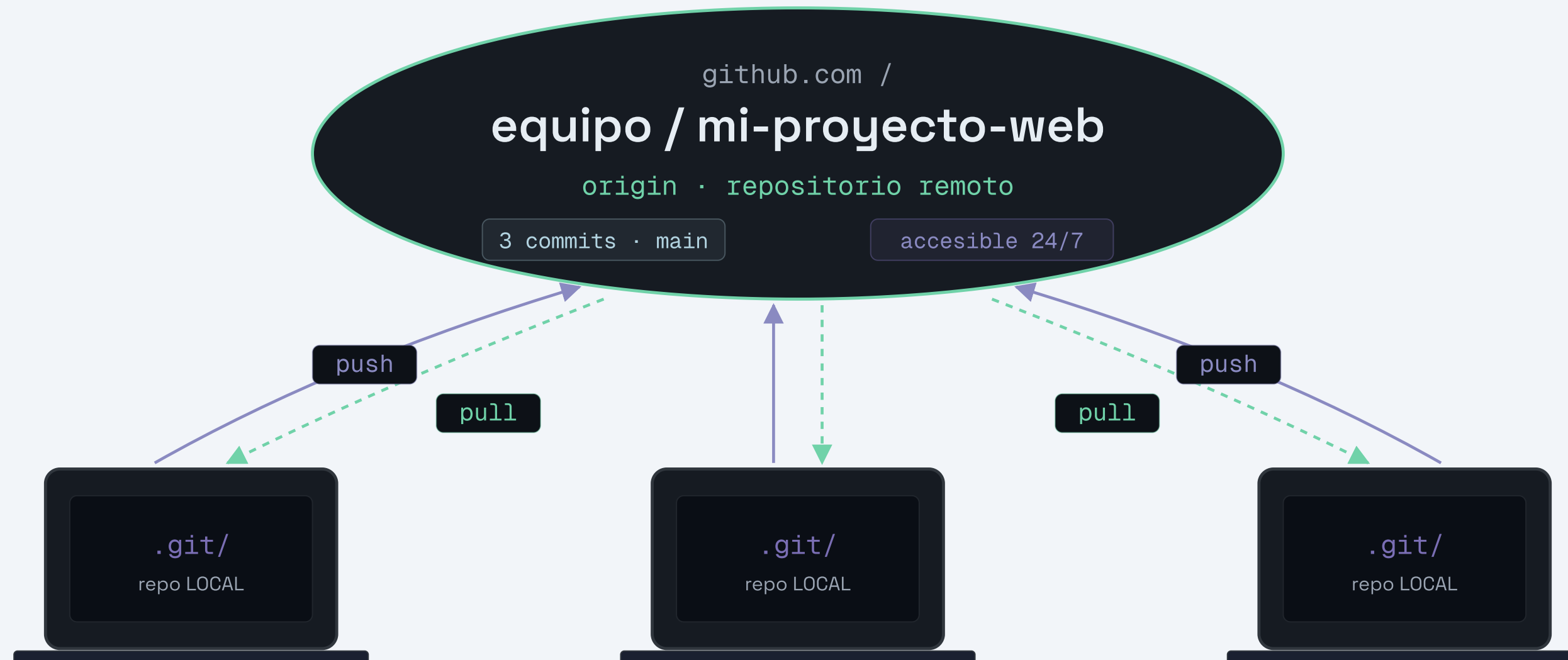
3 Repositorio local

`git push`

4 Repositorio remoto

- La copia de tu repo en GitHub
- Accesible desde cualquier computadora
- Es el respaldo y el punto de colaboración del equipo
- Se llama `origin` en los comandos

Una URL • muchas copias sincronizadas



El remoto es el **árbitro**: los locales se sincronizan con él, no entre sí.

Crear el primer repositorio

```
# 1. Ir al escritorio
$ cd Desktop

# 2. Crear la carpeta del proyecto
$ mkdir mi-proyecto-web

# 3. Entrar a la carpeta
$ cd mi-proyecto-web

# 4. Inicializar el repositorio
$ git init
```

RESULTADO ESPERADO

```
Initialized empty Git repository in .../mi-proyecto-web/.git/
```

Repositorio — referencia rápida

Inicializar repo en la carpeta actual

git init

Ver la carpeta oculta .git

ls -la *# Mac/Linux*

dir /a *# Windows*

Navegar la terminal

pwd *# ¿Dónde estoy?*

ls / **dir** *# ¿Qué hay aquí?*

cd nombre *# Entrar*

cd .. *# Regresar un nivel*

mkdir nombre *# Crear carpeta*

Las tripas de tu repositorio

```
.git/
├── HEAD           # dónde estás parado
├── config         # config local del repo
├── description    # (legacy · sin uso)
├── hooks/         # scripts automáticos
├── info/exclude   # gitignore local privado
├── objects/       # commits · árboles · blobs
│   ├── 1a/e6f4...
│   ├── e6/3d92...
│   └── pack/
├── refs/          # punteros a commits
│   ├── heads/main # tu rama local
│   └── tags/       # etiquetas de versión
└── logs/          # historial de movimientos
```

objects/

El corazón de Git. Cada commit, cada versión de cada archivo se guarda aquí como un objeto identificado por su **hash SHA-1** — los dos primeros caracteres son la subcarpeta.

refs/heads/

Cada archivo aquí es una rama. Contiene el hash del último commit de esa rama. Por eso una rama en Git es tan barata: es solo un archivo de texto con un hash adentro.

HEAD

Apunta a la rama (o commit) en la que estás trabajando ahora. Es lo que cambia cuando haces `git checkout`.

Regla nuclear: no edites `.git/` a mano. Si la borras, perdiste el historial. Si la editas mal, corrompes el repo.

- EL ARCHIVO `.GITIGNORE`

Decirle a Git: "esto NO lo cuentas"

`.gitignore` es un archivo de texto en la raíz del proyecto donde escribes los patrones de archivos que Git debe ignorar. No aparecen en `git status`, no se pueden agregar al staging, no llegan a GitHub.

... `.gitignore`

```
# dependencias (se reinstalan)
node_modules/
vendor/

# builds y archivos generados
dist/
build/
*.log

# archivos del sistema operativo
.DS_Store
Thumbs.db

# archivos del editor
.vscode/
.idea/

# secretos - NUNCA subir
.gitignore
.env
```

Patrones útiles

`*.log` todos los `.log`
`build/` esta carpeta entera
`/secreto.txt` solo en la raíz
`!important.log` excepción (no ignorar)

¿Dónde lo creo?

En la **raíz del proyecto**, al lado de `README.md`. Se llama exactamente `.gitignore` — con punto, sin extensión.

Si ya hiciste commit...

Agregarlo al `.gitignore` **no lo borra** del historial. Remuévelo del tracking con:

```
git rm --cached archivo
```

Tu primer repositorio

QUÉ SE PIDE

- Crea una carpeta para tu proyecto y entra en ella
- Inicializa el repositorio con `git init`
- Explora la carpeta `.git/` con `ls -la`
- Crea un `.gitignore` que ignore `node_modules/` y archivos `.env`

QUÉ SE ENTREGA

- Captura de terminal mostrando `git init` exitoso y el contenido del directorio
- Contenido de tu archivo `.gitignore`

¿Qué cubrimos?

Conceptos clave

- Un repositorio = la carpeta `.git/` oculta dentro de tu proyecto
- Las 4 capas: **Working Dir → Staging → Local Repo → Remote**
- **.gitignore** evita que archivos privados o generados lleguen al historial

Comandos vistos

```
git init
ls -la          # ver .git/
# .gitignore:
node_modules/
.env
```

Checkpoint — ¿tienes tu carpeta de proyecto inicializada con `.gitignore` listo?

PARTE 3

Ciclo de vida de un archivo

- PARTE 3 · OBJETIVOS DE APRENDIZAJE

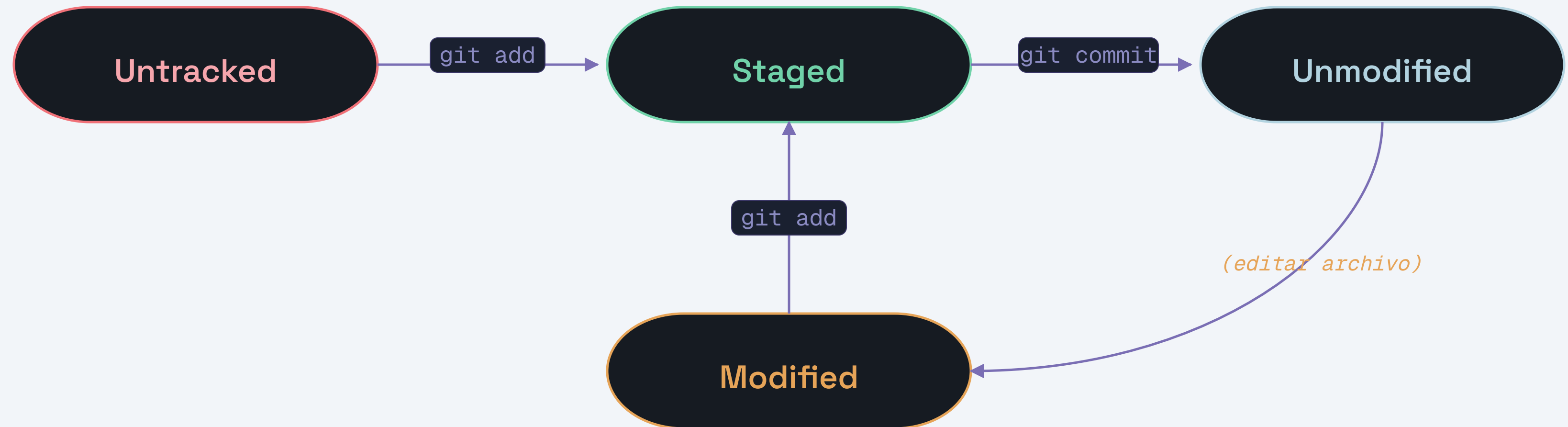
En esta parte aprenderás a...

- RECORDAR — **enumerar** los cuatro estados de un archivo en Git: untracked, staged, unmodified y modified
- COMPRENDER — **describir** el flujo untracked → staged → committed e **interpretar** la salida de `git status`
- APLICAR — **ejecutar** `git add` y `git commit` con mensajes en formato Conventional Commits

En esta sección...

- Conocer los 4 estados de un archivo en Git
- Mover archivos al Staging Area con `git add`
- Guardar un snapshot con `git commit` y Conventional Commits
- Revisar el historial con `git log`

¿En qué estado está tu archivo?



Este es el mapa del bloque. Volveremos a este diagrama en cada estado.

Untracked — sin rastrear

- Git ve el archivo pero **no lo sigue**
- Aparece en **rojo** en `git status`
- Recién creado, o nunca rastreado
- Comando para salir: `git add`

Untracked

→ `git add` →

Staged

Tu primer archivo en el repositorio

ABRE VS CODE EN LA CARPETA

```
$ code .
```

Crea `README.md` y guarda con `Ctrl+S`. Vuelve a la terminal.

CONTENIDO SUGERIDO

```
# Mi proyecto web

**Alumno:** Tu Nombre Completo
**Equipo:** Nombre del equipo
**Proyecto:** Descripción breve
**Curso:** Programación Web · ENP6 UNAM · 2026
```

- EJECUTAR • EL COMANDO MÁS IMPORTANTE

El comando más importante

```
... mi-proyecto-web
```

```
$ git status
```

RESULTADO ESPERADO

```
Untracked files:
  (use "git add <file>..." to include in what will be committed)
  README.md
```

Regla de oro: ejecutar `git status` antes de cualquier otro comando. Siempre.

Staged — preparado

- El archivo está en la **Staging Area**
- Listo para el próximo commit
- Aparece en **verde** en `git status`
- Comando para salir: **git commit**

Staged

→ `git commit` →

Unmodified

Mover al Staging Area

```
... mi-proyecto-web
```

```
$ git add README.md  
$ git status
```

RESULTADO ESPERADO

```
Changes to be committed:  
  new file:   README.md
```

El archivo aparece en verde → estado: **Staged**.

Unmodified — sin cambios

- Archivo registrado y sin cambios desde el último commit
- No aparece en `git status` — todo tranquilo
- Es el estado «en paz»
- Sale de aquí al [editar el archivo](#)

```
# Cuando todo está en orden:  
$ git status  
→ nothing to commit, working tree clean
```

Commit — una fotografía del proyecto

Registro **permanente** del estado del proyecto en un momento exacto.

```
commit a3f8c21
```

```
Author: Sofía López <[email_protected]>
```

```
Date: Sat May 17 11:42:18 2026
```

```
Agrega formulario de inscripción
```

- Hash único: **a3f8c21**
- Mensaje descriptivo
- Autor y fecha
- Exactamente qué cambió, línea por línea

Es **inmutable** — una vez hecho, no cambia.

El mensaje importa

ANATOMÍA DEL MENSAJE

`type(scope):` descripción breve en infinitivo

*Descripción más larga (opcional) explicando qué cambió y por qué.
Una línea en blanco separa título y cuerpo.*

TIPOS MÁS COMUNES

feat	nueva funcionalidad
fix	corrección de bug
docs	documentación
style	formato, espacios, sin cambio de lógica
refactor	reorganizar código sin cambiar comportamiento
test	añadir o corregir pruebas
chore	mantenimiento, dependencias
perf	mejora de rendimiento

MAL MENSAJE

cambios

fix

listo

asdfgh

BUEN MENSAJE

feat(form): agrega
formulario de inscripción

fix(db): corrige conexión
en db.php

style(navbar): elimina
estilos duplicados

docs(readme): actualiza
instrucciones

Máximo 72 caracteres en el título • verbo en infinitivo • en español.

Cada commit es una fotografía del proyecto



Un commit = **hash** + **autor** + **fecha** + **mensaje** + **snapshot de archivos** + **enlace al padre**.

La primera fotografía

```
... mi-proyecto-web
```

```
$ git commit -m "feat(readme): agrega descripción inicial del proyecto"
```

RESULTADO ESPERADO

```
[main (root-commit) f91d340] feat(readme): agrega descripción inicial...  
1 file changed, 6 insertions(+)  
create mode 100644 README.md
```

Modified — modificado

- El archivo fue editado después del último commit
- Git detectó los cambios
- Aparece en rojo en `git status`
- Comando para salir: `git add`

Demostración: editar una línea del README → guardar → `git status` → verlo en rojo.

Ciclo de vida — referencia rápida

```
# Ver el estado de los archivos
git status

# Agregar al Staging Area
git add archivo.txt
git add .                # todos

# Registrar los cambios
git commit -m "feat(scope): descripción breve"

# Ver el historial
git log
git log --oneline
git log --oneline -5
```

ESTADO	EN GIT STATUS	AVANZAR CON
Untracked	Rojo · untracked files	git add
Staged	Verde · changes to be committed	git commit
Unmodified	No aparece	(editar)
Modified	Rojo · changes not staged	git add

Tu primer commit

QUÉ SE PIDE

- Crea un archivo `README.md` con tu nombre y descripción del proyecto
- Agrégalo al staging: `git add README.md`
- Haz tu primer commit con mensaje en `Conventional Commits`
- Verifica con `git status` y `git log --oneline`

QUÉ SE ENTREGA

- Captura de `git log --oneline` con al menos 1 commit
- El mensaje del commit debe seguir el formato `Conventional Commits`

¿Qué cubrimos?

Conceptos clave

- El ciclo: **untracked** → **staged** → **committed** → **modified**
- Un commit es una **foto completa** del proyecto — no solo las diferencias
- Conventional Commits: `tipo(scope): descripción`

Comandos vistos

```
git status
git add README.md
git add .
git commit -m "feat: descripción"
git log --oneline
```

Checkpoint — ¿tienes al menos un commit con mensaje en Conventional Commits?

PARTE 4

Conectar con GitHub

En esta parte aprenderás a...

- RECORDAR — **mencionar** los comandos para conectar un repositorio local con un remoto en GitHub
- COMPRENDER — **distinguir** la diferencia entre `push` y `pull`, y por qué GitHub ya no acepta contraseñas
- APLICAR — **publicar** tu repositorio en GitHub con `git remote add` y `git push` autenticado con un PAT

En esta sección...

- Crear un repositorio remoto en [GitHub](#)
- Conectar tu repo local con `git remote add origin`
- Subir tus commits con `git push` y configurar credenciales (PAT)
- Descargar cambios con `git pull` y clonar con `git clone`

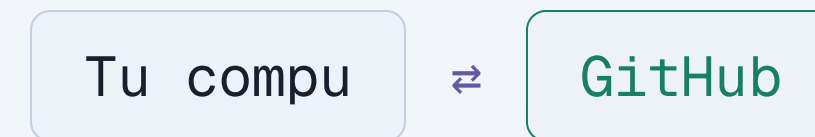
Tu repositorio local no es suficiente

PROBLEMA

- Si se daña la computadora → pierdes todo
- Tu equipo no puede ver tu trabajo
- No tienes respaldo

SOLUCIÓN

Conectar el repositorio local con **GitHub**.



Seguro • compartido • respaldo

Crear el repositorio en GitHub

- 1 Ir a github.com → iniciar sesión
- 2 Clic en + → [New repository](#)
- 3 Nombre: [mi-proyecto-web](#)
- 4 Descripción breve del proyecto
- 5 Sin inicializar con README — ya tienes uno
- 6 Clic en [Create repository](#)
- 7 Copiar la URL: <https://github.com/tu-usuario/mi-proyecto-web.git>

Conectar local con remoto

```
... mi-proyecto-web
```

```
$ git remote add origin https://github.com/tu-usuario/mi-proyecto-web.git  
$ git remote -v
```

RESULTADO ESPERADO

```
origin https://github.com/tu-usuario/mi-proyecto-web.git (fetch)  
origin https://github.com/tu-usuario/mi-proyecto-web.git (push)
```

`origin` es el nombre que le damos al repositorio remoto.

GitHub bloqueó las contraseñas en agosto 2021

EL PROBLEMA CON CONTRASEÑAS

- Una contraseña da **acceso total** a tu cuenta si se filtra
- Git la guardaba en texto plano en la memoria del sistema
- Ataques de phishing y credential stuffing son comunes
- En agosto 2021 GitHub la **desactivó definitivamente** para operaciones Git

Si intentas usar tu contraseña recibes: remote: Support for password authentication was removed

LAS DOS ALTERNATIVAS

Personal Access Token (PAT)

Una cadena larga tipo ghp_xK9... que actúa como contraseña. Se genera en GitHub, tiene **permisos acotados** y fecha de expiración. **Lo que vamos a usar hoy.**

SSH Keys

Par de claves criptográficas (pública + privada). Sin fecha de expiración, más seguro a largo plazo. Requiere configuración extra. *Opción avanzada para después.*

Generar tu Personal Access Token

- 1 Clic en tu foto de perfil → Settings
- 2 Menú lateral → baja hasta Developer settings
- 3 Abre Personal access tokens → Tokens (classic)
- 4 Clic en Generate new token (classic)
- 5 **Note:** `laptop-clase`
- 6 **Expiration:** `90 days`
- 7 Scope: marca `repo` (primera casilla)
- 8 Clic en Generate token

Copia el token AHORA

GitHub lo muestra **una sola vez**. Si cierras la página sin copiarlo, tendrás que generar uno nuevo.

... Tu token

```
ghp_xK9mT2rBqLvN8sYdHwJe  
Pf3CiUoA1gZ5X7kWnRmE
```

Guárdalo en un lugar seguro: en un archivo local que **NO** suba a GitHub, en tu gestor de contraseñas, o en las notas de tu teléfono.

Git pide credenciales — ¿qué escribo?

CUANDO GIT TE PREGUNTE

```
... terminal

$ git push -u origin main
Username for 'https://github.com': tu-usuario-github
Password for 'https://tu-usuario@github.com': ghp_XXXX... ← el PAT
```

En **Password** NO escribas tu contraseña de GitHub — pega el PAT.
No verás los caracteres mientras escribes, eso es normal.

EL SISTEMA GUARDA LAS CREDENCIALES

- **Windows — Git Credential Manager**

```
git config --global credential.helper manager
```

Incluido en Git for Windows. Abre un diálogo gráfico la primera vez y guarda en el Administrador de credenciales de Windows.

- **macOS — Llavero del sistema (osxkeychain)**

```
git config --global credential.helper osxkeychain
```

Incluido en macOS. Las credenciales van al Llavero — ver en Keychain

Access → github.com

- **Linux — configura con un comando:**

```
git config --global credential.helper store
```

Guarda en ~/.git-credentials en texto plano — solo en equipo personal

Si el token expira o cambias de computadora, borra la entrada guardada y Git te pedirá las credenciales en el próximo push.

Subir al mundo

```
... mi-proyecto-web
```

```
$ git push -u origin main
```

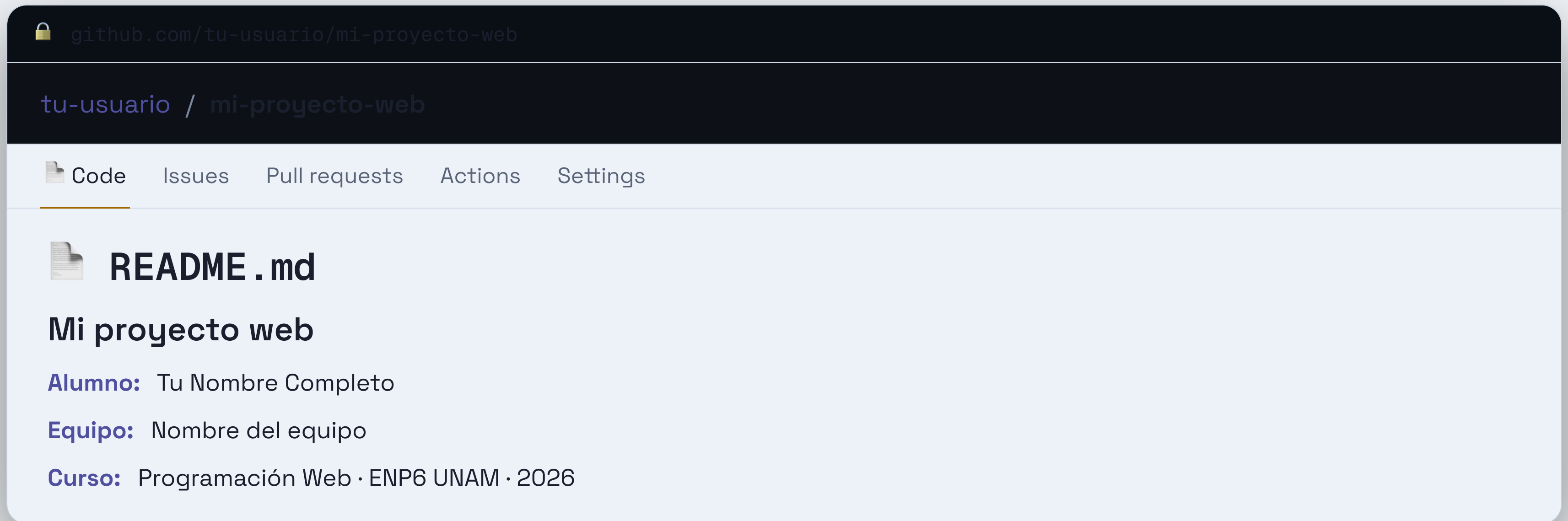
RESULTADO ESPERADO

```
Branch 'main' set up to track remote branch 'main' from 'origin'.
```

Después del primer push, solo necesitas:

```
git push
```

Abre tu navegador



El código que escribiste hace unos minutos ya existe en internet.

Traer los cambios del remoto

```
$ git pull
```

CUÁNDO USARLO

- Antes de empezar a trabajar cada día
- Cuando un compañero hizo push y quieres sus cambios

SI NO HACES PULL ANTES

Riesgo de **conflictos** al hacer push.

Descargar un repositorio completo

```
$ git clone https://github.com/usuario/repositorio.git
```

CUÁNDO USARLO

- Para incorporarte a un proyecto existente
- Cuando tu compañero creó el repo del equipo

DESPUÉS DEL CLONE

Ya no necesitas `git init` ni `git remote add`. Para actualizar después: `git pull`, no clone de nuevo.

Repositorio remoto — referencia rápida

```
# Conectar repo local con GitHub  
git remote add origin https://github.com/usuario/repo.git
```

```
# Ver los remotos configurados  
git remote -v
```

```
# Subir commits al remoto  
git push -u origin main    # primera vez  
git push                  # siguientes
```

```
# Descargar cambios del remoto  
git pull
```

```
# Clonar un repositorio completo  
git clone https://github.com/usuario/repo.git
```

Subir a GitHub

QUÉ SE PIDE

- Crea un repositorio (público) en GitHub
- Conéctalo: `git remote add origin <url>`
- Sube tus commits: `git push -u origin main`
- Configura tu **PAT** en el administrador de credenciales

QUÉ SE ENTREGA

- URL de tu repositorio en **GitHub** con al menos 1 commit visible
- El **README.md** debe verse en la página del repo

¿Qué cubrimos?

Conceptos clave

- **origin** es el alias del repositorio remoto — puedes tener más de uno
- `push` sube commits; `pull` descarga y fusiona; `clone` copia el repo completo
- Un **PAT** reemplaza la contraseña para autenticarte con GitHub desde la terminal

Comandos vistos

```
git remote add origin <url>
git push -u origin main
git pull
git clone <url>
```

Checkpoint — ¿tu repositorio ya aparece en tu perfil de GitHub con commits visibles?

Git — referencia completa • Parte 1

```
# — CONFIGURACIÓN (una sola vez) —————
git config --global user.name "Tu Nombre"
git config --global user.email "[email protected]"
git config --list                # verificar

# — INICIAR UN PROYECTO —————
git init                        # nuevo repositorio local
git clone <url>                 # clonar repositorio existente

# — REVISAR ESTADO —————
git status                     # estado de los archivos
git log                        # historial completo
git log --oneline              # historial compacto
git log --oneline --all --graph # árbol visual

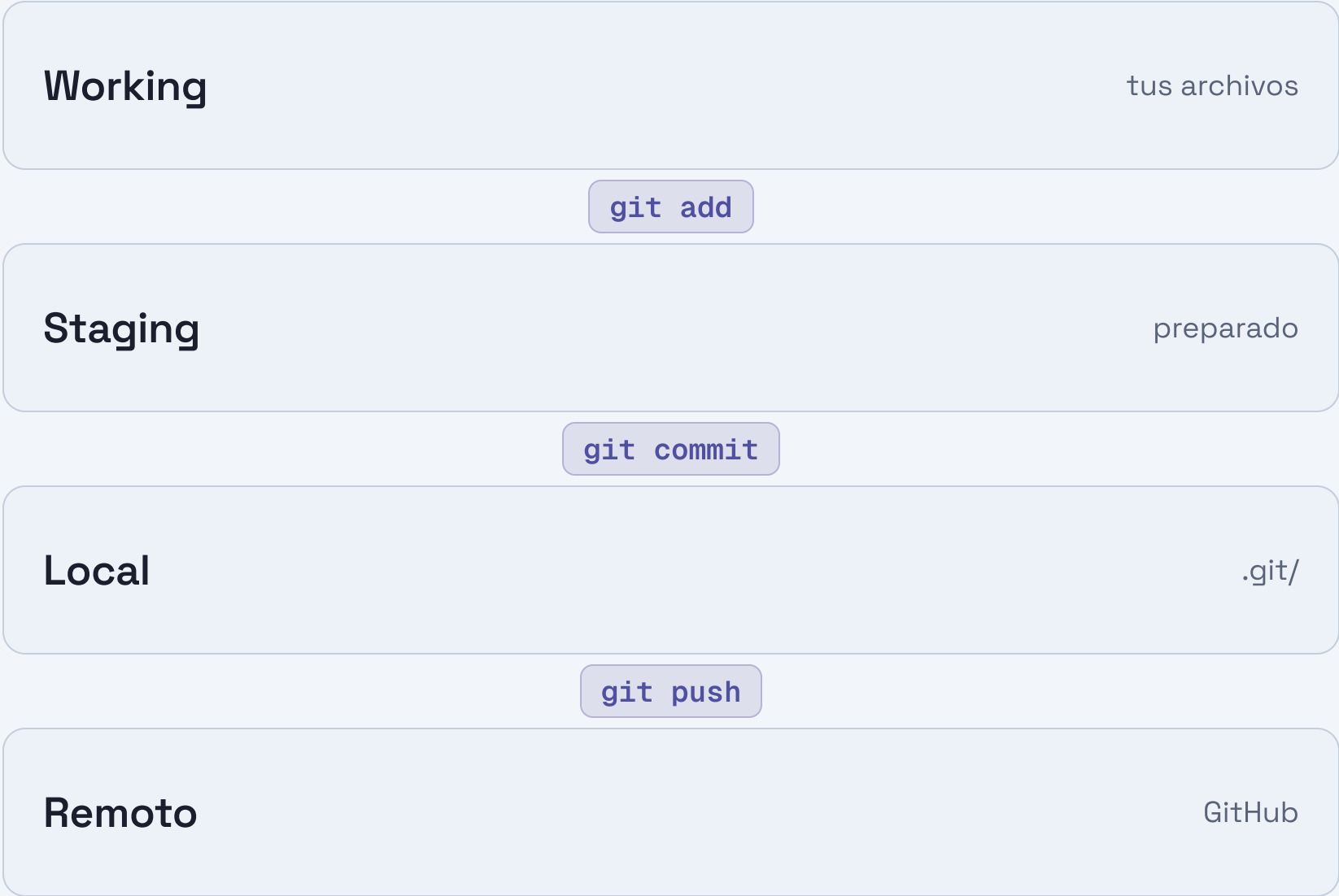
# — CICLO ADD → COMMIT —————
git add <archivo>              # agregar un archivo
git add .                      # todos los cambios
git commit -m "feat(scope): descripción breve" # registrar
```

Git — referencia completa • Parte 2

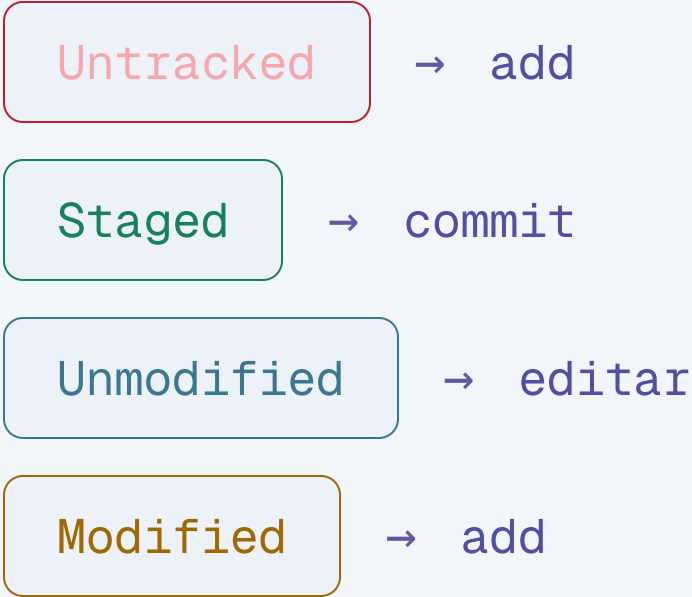
```
# — TRABAJO CON REMOTO —————  
git remote add origin <url>           # conectar con GitHub  
git remote -v                          # ver remotos  
git push -u origin main                 # primera subida  
git push                               # subidas siguientes  
git pull                               # traer cambios  
  
# — RAMAS —————  
git branch                             # ver rama actual  
git branch <nombre>                   # crear rama  
git checkout <nombre>                 # cambiar de rama  
git checkout -b <nombre>              # crear y cambiar  
git merge <nombre>                    # fusionar rama  
  
# — CICLO DE VIDA DEL ARCHIVO —————  
# Untracked → git add → Staged  
# Staged → git commit → Unmodified  
# Unmodified → (editar) → Modified  
# Modified → git add → Staged
```

Lo que ya saben hacer

LAS 4 CAPAS



EL CICLO DE VIDA



PREGUNTA DE SALIDA

¿Cuál es la diferencia entre `git add` y `git commit`?

- PARA SEGUIR PRACTICANDO

Para seguir practicando

RECURSOS

RECURSO	URL
Git Mastery — práctica interactiva	gitmastery.me
Learn Git Branching — visualizador	learngitbranching.js.org
Game4Git — juego en español	game4git.games/es
Learn Markdown Game	learn-markdown-game.com
Git Cheat Sheet oficial	git-scm.com/cheat-sheet

ANTES DE LA SIGUIENTE CLASE

- Tu repo en GitHub con al menos **3 commits** propios
- Mensajes en formato **type(scope): descripción**
- Dibujar en el cuaderno el **ciclo de vida** de un archivo

PREGUNTA DE SALIDA

¿Cuál es la diferencia entre **git add** y **git commit**?

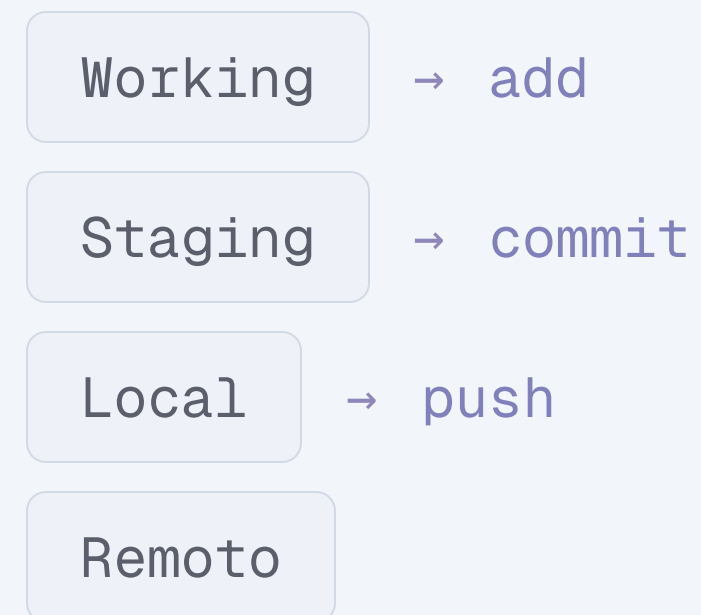
CONTINUAMOS

Trabajo colaborativo con Git

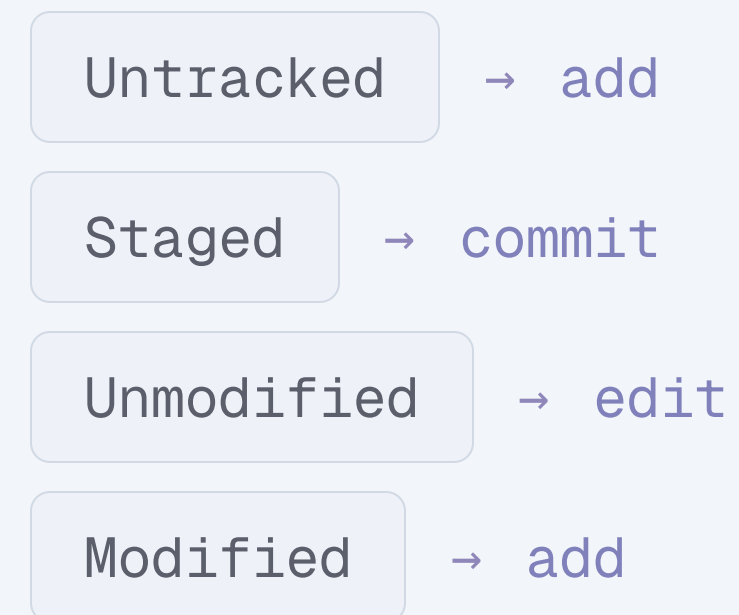
- ¿DÓNDE NOS QUEDAMOS?

Lo que ya saben hacer

LAS 4 CAPAS



CICLO DE VIDA



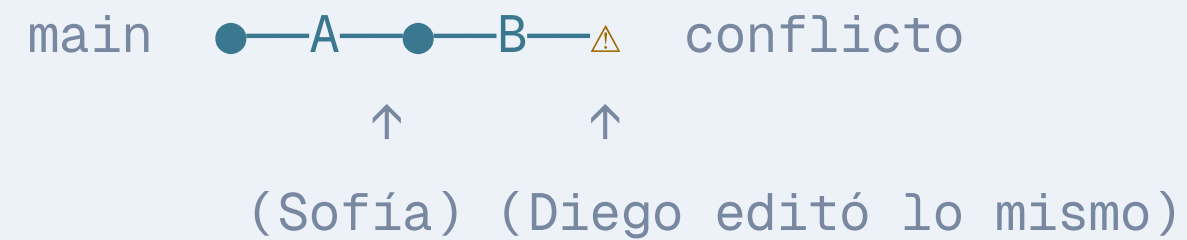
PREGUNTA AL GRUPO

¿Cuál es el primer comando que ejecutan antes de empezar a trabajar cada día?

Apertura Apertura

- EL PROBLEMA DEL DÍA

¿Qué pasa si dos personas editan el mismo archivo en `main` al mismo tiempo?



¿Quién tiene razón? ¿Qué versión queda?

La respuesta: **las ramas**. Ese es el tema de hoy.

Apertura Apertura

PARTE 5

Trabajo en equipo

En esta parte aprenderás a...

- **COMPRENDER** — **explicar** el flujo de colaboración: clone → commit → push en un repositorio compartido
- **APLICAR** — **contribuir** al repositorio del equipo con commits propios y sincronizados con el remoto
- **ANALIZAR** — **interpretar** `git log` para identificar quién cambió qué y en qué orden
- **COMPRENDER** — **reconocer** qué provoca un conflicto de merge y qué estrategias lo evitan

En la sesión anterior dejaron todo listo.

Ahora lo usan en equipo.

- Formar equipos · un integrante es el **owner** del repositorio compartido
- Los demás se incorporan como colaboradores y **clonan el repo del equipo**
- Hacer commits y **push** en paralelo — y ver qué pasa cuando coinciden
- Leer el historial colaborativo con **git log** y entender los conflictos antes de que lleguen

El flujo cuando son varios

Compañero A

Compañero B

Crea el repo en GitHub

`git push`

Edita archivos

`git add → commit → push`

`git pull` →

← `git clone` (una vez)

← `git pull`

Edita archivos

`git add → commit → push`

Regla de equipo: pull → trabajar → add → commit → push → repetir.

Crear el repositorio compartido

- 1 Decidan quién es el **owner** — esa persona **crea un repositorio nuevo** en GitHub con el nombre del proyecto
- 2 Va a **Settings → Collaborators** → busca a sus compañeros por usuario y los invita
- 3 El resto **acepta la invitación** que llega al correo — sin esto no pueden hacer push

Esperen a que todos acepten la invitación antes de continuar.

Todos tienen el repo en su máquina

LOS QUE NO CREARON EL REPO

```
git clone https://github.com/equipo/nombre-proyecto.git
cd nombre-proyecto
```

EL QUE CREÓ EL REPO

```
# Ya tiene el repo, solo verifica
git pull
```

Cada quien firma el proyecto

Cada integrante edita el [README.md](#) y agrega su nombre. Luego el flujo de siempre:

```
●●● nombre-proyecto
```

```
$ git add README.md  
$ git commit -m "feat(readme): agrega nombre de [Tu Nombre] al equipo"  
$ git push
```

Si hay error al hacer push: primero `git pull` para traer los commits de los compañeros.

El historial ya es de todos

```
... nombre-proyecto
```

```
$ git log --oneline
```

```
d2a1f98 Agrega nombre de Sofía al equipo  
c7b3e21 Agrega nombre de Miguel al equipo  
b4f9d12 Agrega nombre de Karla al equipo  
a8c2e07 Agrega nombre de Diego al equipo  
f91d340 Primer commit: agrega README con descripción
```

Pregunta: ¿qué información da cada línea? ¿quién hizo más cambios? ¿qué dicen los mensajes?

• REFLEXIÓN

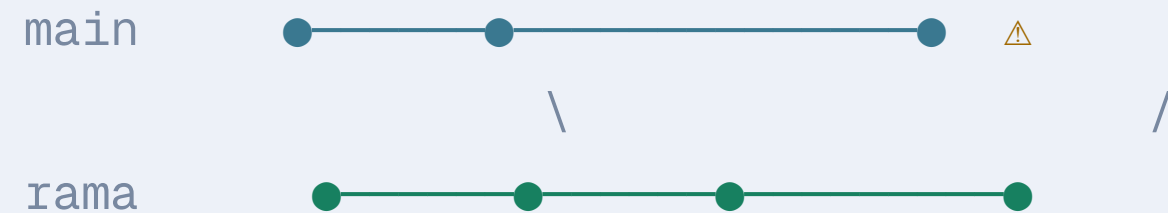
El historial no miente

“ El log es la evidencia de quién trabajó y cuándo. No importa quién habla en la presentación — Git ya tiene el registro. ”

- QUÉ PUEDE PASAR

Conflictos de merge

- Ocurre cuando **dos personas editan la misma línea** del mismo archivo y ambas hacen push
- Git lo detecta, rechaza el push y **pide que decidan manualmente** qué versión queda
- En este ejercicio: cada quien edita su propia sección del archivo — así no se pisan
- En la Parte 6 — que viene a continuación — los resuelven en vivo



Flujo de trabajo en equipo — referencia rápida

```
# El flujo de cada día de trabajo
git pull                    # 1. Traer los últimos cambios
# ... editar archivos ...
git status                  # 2. Ver qué cambió
git add .                   # 3. Preparar los cambios
git commit -m "feat(scope): descripción breve" # 4. Fotografiar el estado
git push                    # 5. Subir al equipo

# Ver qué hicieron los compañeros
git log --oneline
git log --oneline --all --graph # vista de árbol con ramas
```

Regla de equipo: **git pull** antes de empezar. Siempre.

Branch — una línea paralela de desarrollo

Una **copia independiente** del proyecto donde trabajas **sin tocar** lo que ya funciona en `main`.

PARA QUÉ SIRVE

- Cada persona desarrolla su parte **sin romper main**
- El equipo avanza en paralelo — sin esperar a que el otro termine
- `main` siempre tiene el código estable; las ramas tienen el trabajo en progreso

- DIAGRAMA

Así se ven las ramas



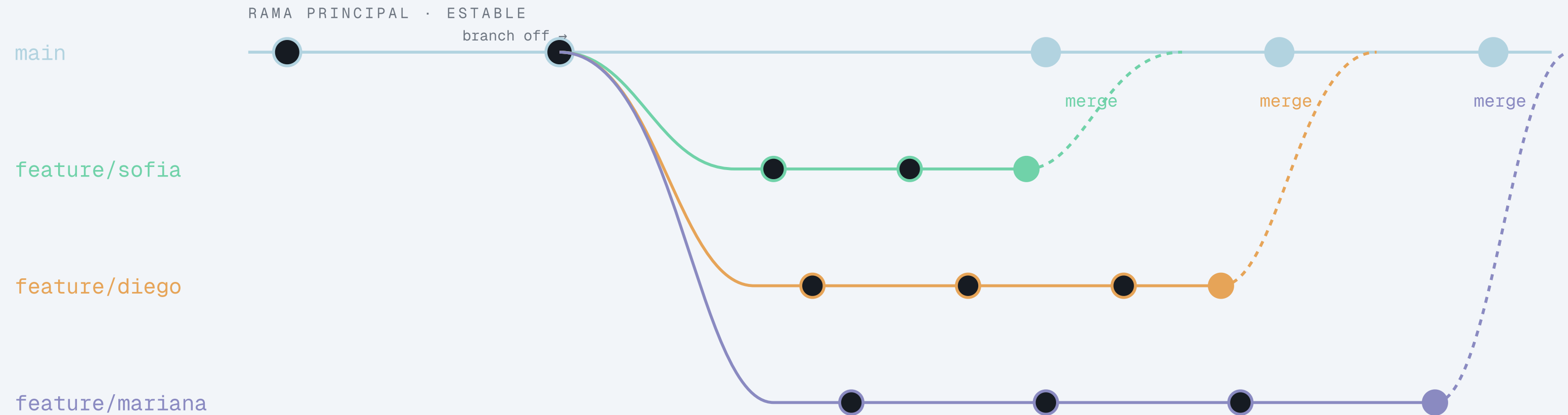
● = commit

main = rama principal · código estable

feature = rama de desarrollo paralelo

En unos minutos crean su primera rama — guarden esta imagen en la cabeza.

3 personas • 3 ramas • una main



Cada persona **trabaja aislada** en su rama • al terminar la **fusiona** a **main**.

- RAMAS • LA OTRA MITAD DEL CONCEPTO

Merge — cuando los caminos se unen

SIN CONFLICTOS



CON CONFLICTOS



Los comandos que van a usar ahora

Ver en qué rama estás

`git branch`

Crear una nueva rama

`git branch nombre-rama`

Cambiar de rama

`git checkout nombre-rama`

Crear y cambiar en un solo comando

`git checkout -b nombre-rama`

Fusionar una rama con la actual

`git merge nombre-rama`

Ver todas las ramas (local y remota)

`git branch -a`

Guarden esta referencia — en la siguiente sección la usan en vivo.

¿Qué cubrimos?

Conceptos clave

- Para colaborar: un integrante es **owner**, los demás hacen clone y se agregan como colaboradores
- `git log` es el historial compartido — cada commit tiene autor, fecha y hash
- Un **conflicto** ocurre cuando dos personas editan las mismas líneas — `pull` antes de push lo reduce

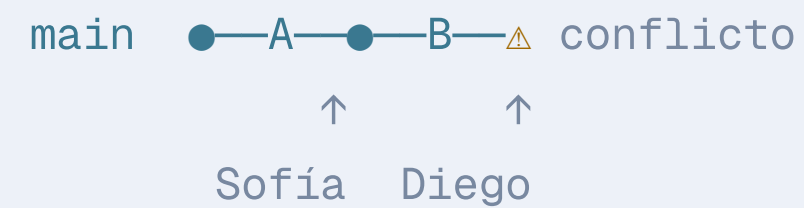
Comandos vistos

```
git clone <url>
git pull
git log --oneline --graph
git log --author="Nombre"
```

Checkpoint — ¿el historial de `git log` muestra commits de al menos dos integrantes del equipo?

Sin ramas • Con ramas

SIN RAMAS — TODOS EN MAIN



CON RAMAS — CADA QUIEN EN LA SUYA



- CONCEPTO

Branch — una copia de trabajo independiente

Una **línea de desarrollo paralela** que no afecta a **main** hasta que tú decides fusionarla.

EN EL PROYECTO DEL CURSO

- **main** = versión estable del proyecto del equipo
- **feature/nombre** = espacio de trabajo personal de cada integrante
- Nadie trabaja directamente en main a partir de hoy

Cómo nombrar las ramas

TIPOS DE RAMA

PREFIJO	CUÁNDO USARLA
main	Rama principal • siempre estable
feature/	Funcionalidad nueva
bugfix/	Corrección de bug no urgente
hotfix/	Corrección urgente en producción
release/	Preparar una versión
chore/	Mantenimiento, refactor, dependencias

REGLAS DE NOMBRADO

- Solo minúsculas y guiones — sin espacios, acentos, underscores ni mayúsculas
- Sin guiones al inicio, al final, ni consecutivos
- Conciso y descriptivo
- Si hay un ticket, incluirlo: feature/123-add-login

EJEMPLOS VÁLIDOS

feature/add-login-page
bugfix/fix-header-bug
hotfix/update-broken-link
release/v1.0.1
chore/update-dependencies

En este curso: cada integrante tiene su propia rama con formato `feature/nombre-integrante` (p. ej. `feature/sofia`).

Ramas — comandos esenciales

Ver en qué rama estás

```
git branch
```

Ver todas las ramas (local y remota)

```
git branch -a
```

Crear una nueva rama

```
git branch feature/sofia
```

Cambiar a una rama existente

```
git checkout feature/sofia
```

Crear y cambiar en un solo comando (el más usado)

```
git checkout -b feature/sofia
```

Subir la rama al remoto por primera vez

```
git push -u origin feature/sofia
```

```
git push # siguientes
```

PARTE 6

Ramas y Merge

En esta parte aprenderás a...

- RECORDAR — **identificar** los comandos para crear, cambiar y eliminar ramas
- COMPRENDER — **explicar** por qué trabajar en ramas protege `main` y facilita la revisión de cambios
- APLICAR — **crear** tu rama personal, publicarla en GitHub y fusionarla a `main` con `git merge`
- EVALUAR — **distinguir** cuándo ocurre un conflicto y **resolver** uno básico editando los marcadores de Git

En esta sección...

- Crear tu rama personal con `git checkout -b feature/tu-nombre`
- Hacer commits en tu rama sin afectar `main`
- Subir la rama al remoto con `git push -u origin feature/tu-nombre`
- Hacer `merge` de tu rama a `main` y resolver conflictos

Crear tu rama en el repo del equipo

●●● repo-equipo

```
# Asegurarse de estar en main actualizado
$ git checkout main
$ git pull

# Crear y cambiar a tu rama personal
$ git checkout -b feature/tu-nombre
$ git branch
```

RESULTADO ESPERADO

```
main
* feature/tu-nombre
```

El asterisco * indica la rama en la que estás.

Hacer un cambio en tu rama

Cada integrante agrega una sección al [README.md](#) con su nombre y responsabilidad.

... feature/tu-nombre

```
$ git add README.md
$ git commit -m "docs(readme): agrega responsabilidades de [tu nombre]"
$ git log --oneline
```

Verifica que el commit está **en tu rama** y no en main.

Compartir tu rama con el equipo

```
... feature/tu-nombre
```

```
$ git push -u origin feature/tu-nombre
```

VERIFICAR EN GITHUB

Repositorio → pestaña [Branches](#) → tu rama debe aparecer ahí.

VER TODAS LAS RAMAS

```
git branch -a
```

Ver el árbol de ramas

```
... repo-equipo
```

```
$ git log --oneline --all --graph
```

```
* a3f8c21 (feature/sofia)  Agrega responsabilidades de Sofía  
| * b7d2e09 (feature/diego)  Agrega responsabilidades de Diego  
| * c4f1a38 (feature/karla)  Agrega responsabilidades de Karla  
|/  
* f91d340 (HEAD -> main, origin/main)  Primer commit: agrega README
```

Pregunta: ¿qué ven en este diagrama? ¿cuántas personas han trabajado?

Ramas — referencia rápida

```
# Crear y cambiar de rama (el flujo del proyecto)
git checkout main           # volver a main
git pull                   # actualizar main
git checkout -b feature/nombre # crear tu rama desde main

# Ver ramas
git branch                 # locales
git branch -a             # todas, incluyendo remotas

# Subir rama al remoto
git push -u origin feature/nombre # primera vez
git push                   # siguientes

# Ver el árbol visual de ramas
git log --oneline --all --graph
```

Merge — unir el trabajo de todos en `main`

Integrar los commits de una rama dentro de otra.

EN EL PROYECTO DEL CURSO

- Cada integrante trabaja en su rama
- Cuando termina algo, **lo integra a main**
- `main` siempre refleja el **estado actual del proyecto completo**

- DIAGRAMA

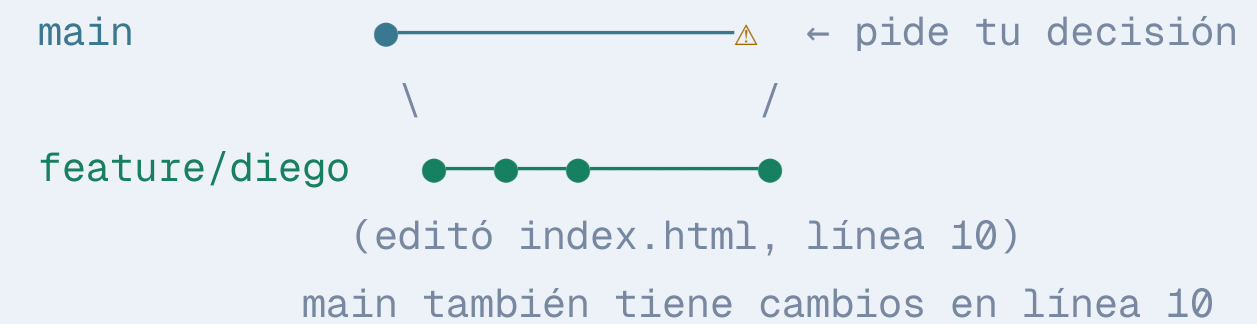
Merge sin conflicto vs con conflicto

A — SIN CONFLICTO (AUTOMÁTICO)



Sofía editó style.css. Nadie más lo tocó. Git fusiona solo.

B — CON CONFLICTO (MANUAL)



Dos personas editaron la misma línea. Git no sabe cuál conservar.

Integrar tu rama a main

● ● ● repo-equipo

```
# 1. Volver a main
$ git checkout main

# 2. Traer los últimos cambios
$ git pull

# 3. Fusionar tu rama con main
$ git merge feature/tu-nombre
```

RESULTADO ESPERADO (SIN CONFLICTO)

```
Updating f91d340..a3f8c21
Fast-forward
 README.md | 3 +++
 1 file changed, 3 insertions(+)
```

Actualizar main en GitHub

● ● ● main

```
$ git push
```

```
$ git log --oneline --all --graph
```

Turno por turno: cada integrante repite los pasos 4 y 5. Antes de cada merge, hacer `git pull` para traer los merges anteriores.

Limpieza — borrar la rama después del merge

● ● ● main

```
# Borrar la rama local  
$ git branch -d feature/tu-nombre  
  
# Borrar la rama en el remoto  
$ git push origin --delete feature/tu-nombre
```

¿POR QUÉ BORRARLA?

- Evita acumular ramas obsoletas
- Mantiene el proyecto ordenado
- En el curso: una rama nueva para cada funcionalidad

Conflicto de merge — no es un error, es una pregunta

Git no puede decidir **automáticamente** qué versión conservar porque dos ramas modificaron **la misma línea** del mismo archivo.

Git no rompe nada — te pregunta:

“Aquí hay dos versiones distintas. ¿Cuál quieres conservar, o cómo las combinas?”

- REFERENCIA

Los marcadores de conflicto

... index.html

```
<<<<<< HEAD (main)
<h1>Portal de inscripciones ENP6</h1>
=====
<h1>Sistema de registro de talleres</h1>
>>>>>> feature/diego
```

- <<<<<< HEAD → versión que está en main
- ===== → separador entre las dos versiones
- >>>>>> feature/diego → versión que viene de la rama

Tu trabajo: decidir qué queda, editar el archivo y quitar los marcadores.

Paso a paso — resolver un conflicto

... repo-equipo

```
# 1. Git avisa del conflicto al hacer merge
$ git merge feature/diego
→ CONFLICT (content): Merge conflict in index.html

# 2. Ver qué archivos tienen conflicto
$ git status
→ both modified: index.html

# 3. Abrir el archivo en VS Code, editar y quitar los marcadores
#     (decidir qué versión queda o combinar ambas)

# 4. Marcar el conflicto como resuelto
$ git add index.html

# 5. Completar el merge
$ git commit -m "chore(merge): resuelve conflicto en index.html"
```

- BUENAS PRÁCTICAS

Prevenir es mejor que resolver

- Cada integrante trabaja en `archivos distintos` siempre que sea posible
- Hacer `git pull` antes de empezar a trabajar `cada día`
- Merges `frecuentes` — no acumular días de trabajo sin integrar
- Comunicar en el equipo `quién está editando qué` archivo
- Commits `pequeños y frecuentes` — más fáciles de integrar que uno grande

Merge y conflictos — referencia rápida

```
# Flujo de merge completo
git checkout main           # ir a main
git pull                   # actualizar main
git merge feature/nombre   # fusionar tu rama
git push                   # subir main actualizado

# Borrar rama después del merge
git branch -d feature/nombre # local
git push origin --delete feature/nombre # remoto

# Si hay conflicto
git status                 # ver archivos en conflicto
# → editar el archivo, quitar marcadores <<<< ==== >>>>
git add archivo-resuelto   # marcar como resuelto
git commit -m "chore(merge): resuelve conflicto en archivo"

# Ver historial con árbol de ramas
git log --oneline --all --graph
```

Tu rama personal → merge a main

QUÉ SE PIDE

- Crea tu rama: `git checkout -b feature/tu-nombre`
- Agrega al menos 1 archivo o cambio y haz `commit`
- Sube la rama: `git push -u origin feature/tu-nombre`
- Haz merge a `main` y resuelve conflictos si los hay
- Borra la rama local y remota tras el merge

QUÉ SE ENTREGA

- Captura de `git log --oneline --all --graph` mostrando la rama y el merge
- URL del repositorio en GitHub con el historial visible

¿Qué cubrimos?

Conceptos clave

- Una **rama** es un puntero ligero a un commit — crearlas es instantáneo y gratuito
- Trabajar en ramas permite que cada integrante avance **sin pisar el trabajo de otros**
- Un conflicto ocurre cuando dos ramas editan las **mismas líneas** del mismo archivo al mismo tiempo

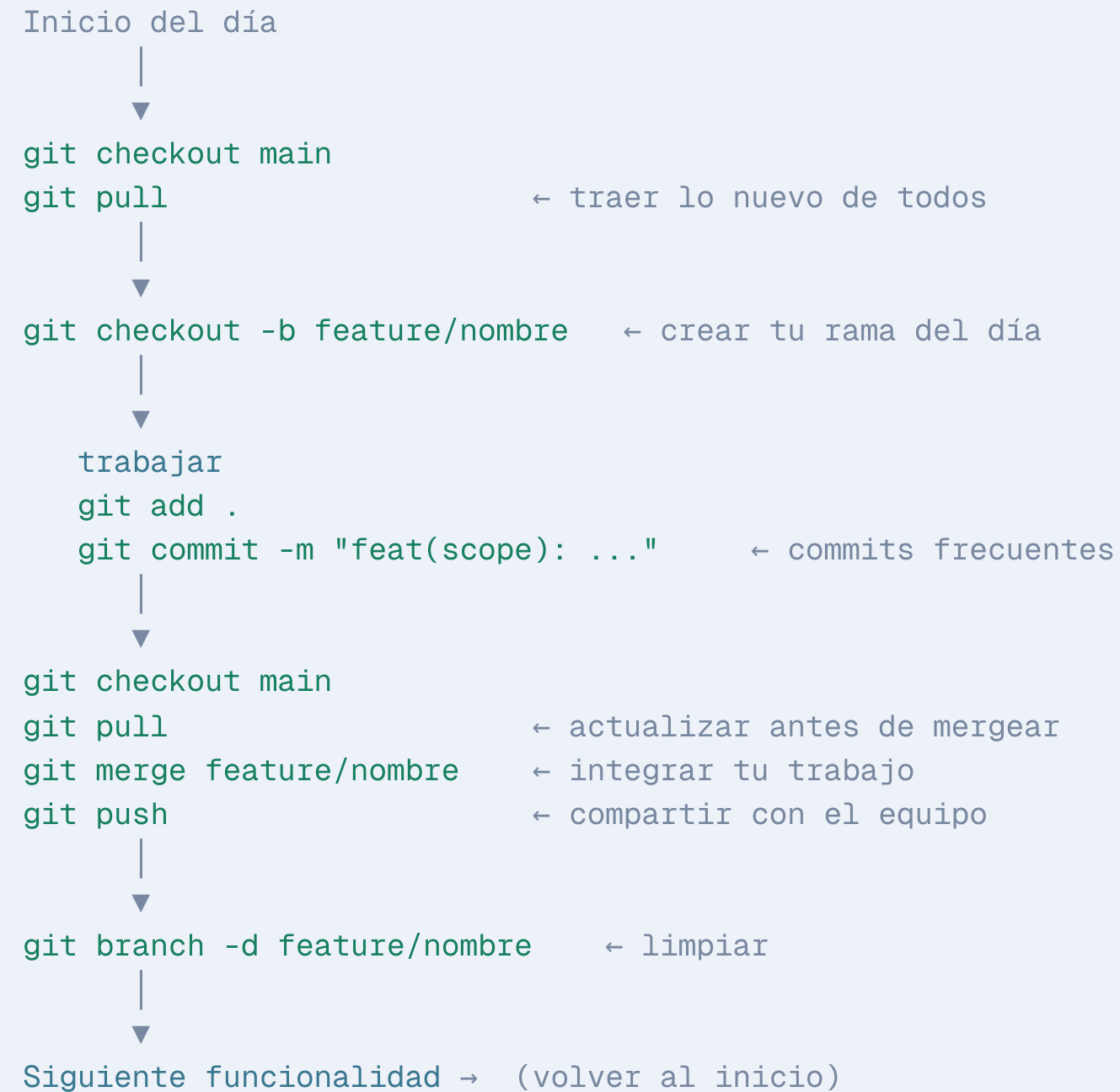
Comandos vistos

```
git checkout -b feature/tu-nombre  
git push -u origin feature/tu-nombre  
git checkout main  
git merge feature/tu-nombre  
git branch -d feature/tu-nombre
```

Checkpoint — ¿tu merge aparece en `git log --graph --oneline` en el repositorio del equipo?

- BLOQUE 3 · FLUJO DEL PROYECTO

Así trabajamos cada día del curso



El acuerdo de trabajo del equipo

Cada equipo agrega una sección al [README.md](#) con sus acuerdos:

```
## Flujo de trabajo del equipo

- Convención de ramas: feature/nombre
- Mensajes: type(scope): descripción
- ¿Cuándo merge? Al terminar cada feature
- ¿Quién resuelve conflictos? Quien hace el merge
- ¿Cada cuánto pull? Al inicio de cada sesión
```

```
## Integrantes y responsabilidades

| Integrante | Rama           | Responsabilidad |
|-----|-----|-----|
| Nombre 1  | feature/nombre1 | HTML           |
| Nombre 2  | feature/nombre2 | CSS            |
| Nombre 3  | feature/nombre3 | JavaScript      |
| Nombre 4  | feature/nombre4 | PHP             |
| Nombre 5  | feature/nombre5 | Base de datos  |
```

```
git add README.md
git commit -m "docs(readme): agrega acuerdos de flujo de trabajo"
git push
```

¿Cómo debe verse su repo hoy?

¿QUÉ DEBE EXISTIR?	CÓMO VERIFICARLO
5 ramas creadas (una por integrante)	<code>git branch -a</code>
Al menos 1 commit por integrante	<code>git log--oneline--all--graph</code>
Todas las ramas mergeadas a main	<code>git log --graph</code> · sin ramas colgantes
README con acuerdos del equipo	Visible en GitHub
main actualizado en GitHub	Ir al repo en github.com

- BONUS CONCEPTUAL • PARA QUE LO SEPAN

Tags • Semantic Versioning

Cuando un proyecto está listo para publicar una versión estable, le ponen una **etiqueta** — un nombre con número. La convención más usada del mundo se llama **SemVer**.

ANATOMÍA DE LA VERSIÓN

v1.4.2

MAJOR cambios incompatibles • rompe lo anterior

MINOR nueva funcionalidad compatible

PATCH corrección de bug compatible

COMANDOS BÁSICOS DE TAGS

```
# Crear un tag con mensaje
git tag -a v1.4.2 -m "Release 1.4.2"

# Ver todos los tags
git tag

# Subir un tag al remoto
git push origin v1.4.2
git push --tags           # todos
```

En este curso no vamos a usar tags — pero en el mundo real, así marcan las versiones de software: **v2.0.0** de React, **v15.3.1** de Node, etc. Es importante reconocer el formato.

Git — referencia completa • Parte 1

```
# — CONFIGURACIÓN (una sola vez) —————
git config --global user.name "Tu Nombre"
git config --global user.email "[email protected]"
git config --list

# — INICIAR UN PROYECTO —————
git init                # nuevo repositorio local
git clone <url>          # clonar repositorio existente

# — REVISAR ESTADO —————
git status              # estado de los archivos
git log --oneline        # historial compacto
git log --oneline --all --graph  # árbol visual

# — CICLO ADD → COMMIT —————
git add <archivo>        # un archivo
git add .                # todos
git commit -m "feat(scope): descripción breve"

# — TRABAJO CON REMOTO —————
git remote add origin <url>
git push -u origin main  # primera subida
git push                 # siguientes
git pull                 # traer cambios
```

Git — referencia completa • Parte 2

```
# — RAMAS —————  
git branch                # ver rama actual  
git branch -a             # todas  
git checkout -b feature/nombre # crear y cambiar  
git checkout main         # volver a main  
git push -u origin feature/nombre # subir rama  
  
# — MERGE —————  
git checkout main  
git pull  
git merge feature/nombre  
git push  
git branch -d feature/nombre # local  
git push origin --delete feature/nombre # remoto  
  
# — CONFLICTOS —————  
git status                # archivos en conflicto  
# editar archivo → quitar marcadores <<<< ==== >>>>  
git add <archivo>         # marcar resuelto  
git commit -m "chore(merge): resuelve conflicto en <archivo>"  
  
# — FLUJO DIARIO DEL EQUIPO —————  
# pull main → crear rama → trabajar → merge → push → limpiar
```

- CIERRE

Para seguir practicando

RECURSOS

RECURSO	URL
Learn Git Branching — visualizador interactivo	learngitbranching.js.org
Git Mastery — práctica de comandos	gitmastery.me
Conventional Commits	conventionalcommits.org
Semantic Versioning	semver.org
Git Cheat Sheet oficial	git-scm.com/cheat-sheet